Лабораторная работа №1

Нейронная сеть прямого распространения с несколькими входами и одним выходным нейроном

Введение в искусственные нейронные сети

Одной из фундаментальных предпосылок в области искусственного интеллекта является предположение о возможности создания машин, способных выполнять задачи, обычно требующие приложения человеческого интеллекта. Мозг человека обладает поразительной способностью обучаться новому.

Почему бы не использовать модель человеческого мозга для создания машины? *Искусственная нейронная сеть* — это модель, предназначенная для имитации процессов обучения, протекающих в мозге человека.

Искусственные нейронные сети проектируются таким образом, чтобы они могли распознавать базовые образы (закономерности, устойчивые взаимосвязи, скрытые в данных) и учиться на них. Они могут применяться для решения различных задач, таких как классификация, регрессия, сегментация данных и др. Прежде чем предоставить данные нейронной сети, мы должны преобразовать их в числовую форму. Например, мы можем иметь дело с данными самой разной природы, включая визуальные и текстовые данные, временные ряды и т.п. Нам приходится принимать решения относительно того, каким образом следует представлять задачи, чтобы они были понятны нейронным сетям.

Создание нейронной сети

Процесс обучения человека носит иерархический характер. В нейронной сети нашего мозга этот процесс осуществляется в несколько стадий, для каждой из которых характерна своя степень гранулярности. На одних стадиях идет обучение простым вещам, на других — более сложным. В качестве примера рассмотрим визуальное распознавание объекта. Когда мы смотрим на ящик, то сначала мы просто идентифицируем такие его элементы, как углы и ребра. На следующей стадии идентифицируется форма ящика, а на последующей — что собой представляет объект. Этот процесс происходит по-разному для различных задач, но, вероятно, идея вам понятна. Создавая такую иерархию, человеческий мозг быстро разделяет понятия и идентифицирует данный объект.

Чтобы имитировать процесс обучения человека, при построении искусственных нейронных сетей используют слои нейронов. Идея этих нейронов подсказана биологическими процессами, о которых шла речь выше. Каждый слой искусственной нейронной сети представляет собой множество независимых нейронов. Каждый нейрон некоторого слоя соединен с нейронами смежного слоя.

Тренировка нейронной сети

Если мы имеем дело с N-мерными входными данными, то входной слой будет состоять из N нейронов. Если среди наших обучающих (тренировочных) данных выделяются M различных классов, то выходной слой будет состоять из M нейронов. Слои, заключенные между входным и выходным слоями, называются *скрытыми*. Простая нейронная сеть состоит из пары слоев, а глубокая нейронная сеть — из множества слоев.

Рассмотрим случай, когда мы хотим использовать нейронную сеть для классификации данных. Первым шагом является сбор подходящих тренировочных данных и их маркирование. Каждый нейрон действует как простая функция, и нейронная сеть тренируется до тех пор, пока ошибка не станет меньше некоторого заданного значения. В качестве ошибки в основном используют разницу между предсказанным и фактическим выходами. Исходя из того, насколько велика ошибка, нейронная сеть корректирует саму себя и повторно обучается до тех пор, пока не приблизится к решению.

мы будем использовать библиотеку NeuroLab, более подробную информацию о которой можно получить на сайте `https://pythonhosted.org/neurolab`. Чтобы установить эту библиотеку, выполните в окне своего терминала следующую команду:

```
$ pip3 install neurolab
```

Создание классификатора на основе перцептрона

Перцептрон — строительный кирпичик искусственной нейронной сети. Он представляет собой одиночный нейрон, который получает входные данные (сигнал), выполняет над ними вычисления и выдает выходной сигнал. Для принятия решений перцептрон применяет простую функцию. Предположим, мы имеем дело с N-мерной входной точкой данных. Перцептрон вычисляет взвешенную сумму N чисел, после чего добавляет к ним константу для получения выходного результата. Эта константа называется *смещением* нейрона. Интересно отметить, что столь простые перцептроны используются для проектирования очень сложных глубоких нейронных сетей. Рассмотрим пример создания классификатора на основе перцептрона с использованием библиотеки NeuroLab.

Создайте новый файл Python и импортируйте следующие файлы.

```
import numpy as np
import matplotlib.pyplot as plt
import neurolab as nl
```

Загрузим входные данные из текстового файла `data_perceptron.txt`. Каждая его строка содержит числа, разделенные пробелом, из которых первые два числа — признаки, а последнее — метка (маркер).

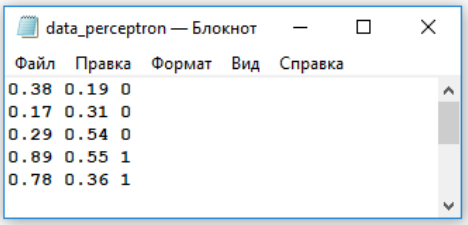

Рис. 14. Файл data_perceptron.txt

```
# Загрузка входных данных
text = np.loadtxt('data_perceptron.txt')
```

Разделим текст на точки данных и метки.

```
# Разделение точек данных и меток
data = text[:, :2]
labels = text[:, 2].reshape((text.shape[0], 1))
```

Построим график точек данных.

```
# Построение графика входных данных
plt.figure()
plt.scatter(data[:,0], data[:,1])
plt.xlabel('Размерность 1')
plt.ylabel('Размерность 2')
plt.title('Входные данные')
```

Определим максимальное и минимальное значения, которые могут достигаться в каждом измерении.

```
# Определение максимального и минимального значений
# для каждого измерения
dim1_min, dim1_max, dim2_min, dim2_max = 0, 1, 0, 1
```

Поскольку данные разделены на два класса, для представления выходного результата требуется всего один бит. В результате выходной слой будет содержать только один нейрон.

```
# Количество нейронов в выходном слое
num_output = labels.shape[1]
```

В нашем наборе точки данных — двумерные. Определим перцептрон с двумя входными нейронами, по одному для каждого измерения.

```
# Определение перцептрона с двумя входными нейронами (поскольку
# входные данные — двумерные)
dim1 = [dim1_min, dim1_max]
dim2 = [dim2_min, dim2_max]
perceptron = nl.net.newp([dim1, dim2], num_output)
```

Обучим перцептрон с помощью тренировочных данных.

```
# Тренировка перцептрона с использованием наших данных
error_progress = perceptron.train(data, labels, epochs=100,
                                  show=20, lr=0.03)
```

Отобразим график процесса обучения, используя метрику ошибки.

```
# Построение графика процесса обучения
plt.figure()
plt.plot(error_progress)
plt.xlabel('Количество эпох')
plt.ylabel('Ошибка обучения')
plt.title('Изменение ошибки обучения')
plt.grid()

plt.show()
```

Полный код примера содержится в файле `perceptron_classifier.py`. В процессе выполнения кода на экране отобразятся два графика. На первом снимке экрана представлены входные точки данных (рис. 14.1).

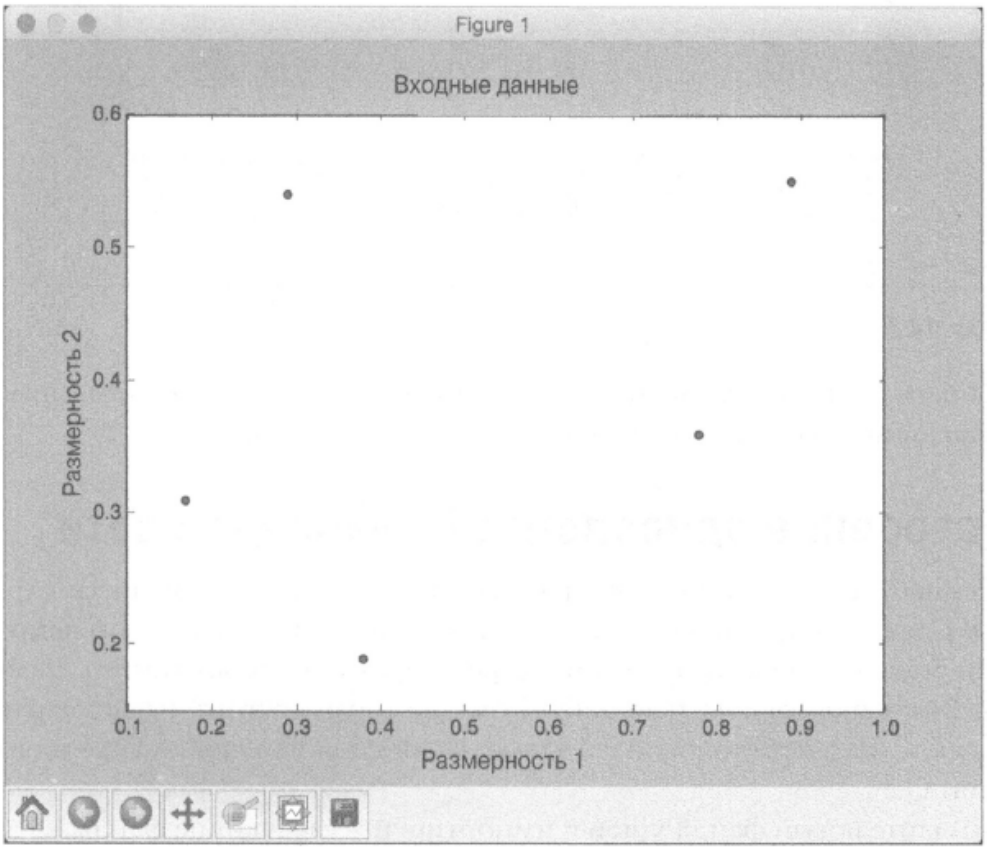

Рис. 14.1

На втором снимке экрана представлена степень продвижения процесса обучения, оцениваемая по величине ошибки (рис. 14.2).

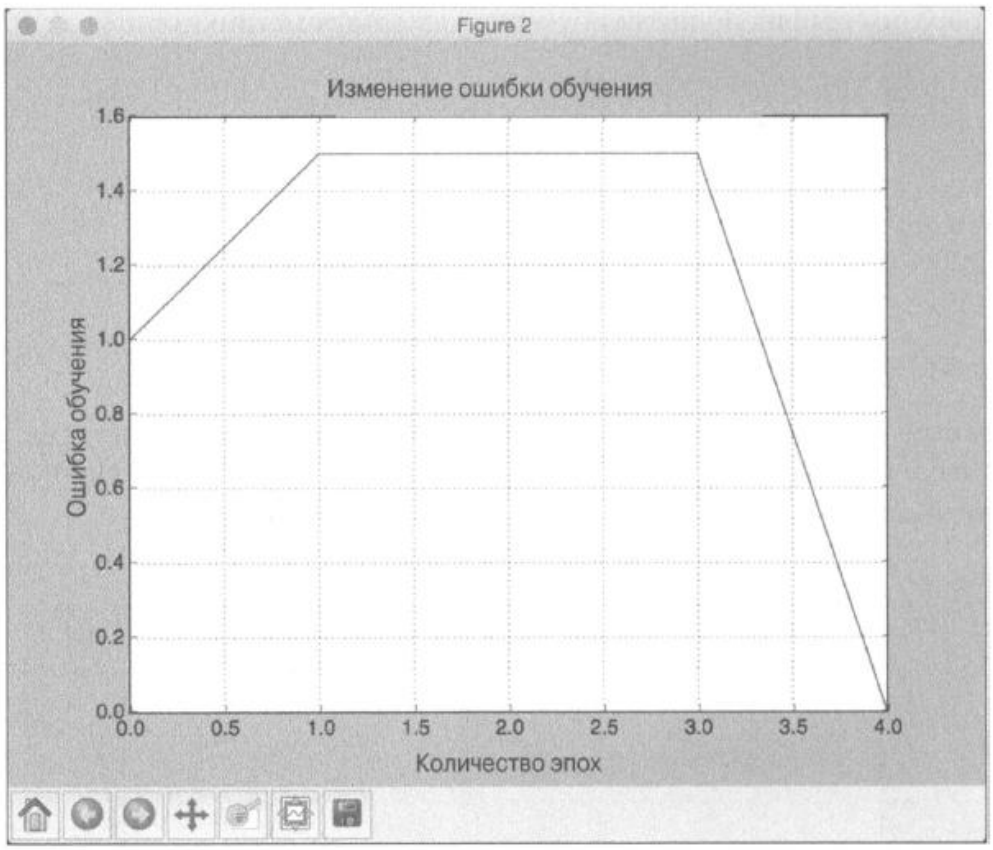

Рис. 14.2

Обратите внимание на то, что в соответствии с предыдущим снимком экрана ошибка снижается до 0 в конце четвертой эпохи.

Содержание работы.

Ответьте на вопросы и выполните следующие задания:

1. **Что называется искусственной нейронной сетью?**
2. **Какие слои нейронной сети называются скрытыми?**
3. **Что называют смещением нейрона?**
4. **Дайте определение “перцептрон”.**
5. **Возможно ли проектирование сложных глубоких нейронных сетей на основе перцептрона?**
6. **Привести код на Python классификатора на основе перцептрона.**
7. **Самостоятельно провести эксперименты с перцептроном на иных произвольных входных данных (файл data_perceptron.txt). Проследить, как изменяется ошибка обучения в зависимости от числа наблюдаемых эпох.**

```python
import numpy as np
import matplotlib.pyplot as plt
import neurolab as nl

# Load input data
text = np.loadtxt('data_perceptron.txt')

# Separate datapoints and labels
data = text[:, :2]
labels = text[:, 2].reshape((text.shape[0], 1))

# Plot input data
plt.figure()
plt.scatter(data[:,0], data[:,1])
plt.xlabel('Dimension 1')
plt.ylabel('Dimension 2')
plt.title('Input data')

# Define minimum and maximum values for each dimension
dim1_min, dim1_max, dim2_min, dim2_max = 0, 1, 0, 1

# Number of neurons in the output layer
num_output = labels.shape[1]

# Define a perceptron with 2 input neurons (because we
# have 2 dimensions in the input data)
dim1 = [dim1_min, dim1_max]
dim2 = [dim2_min, dim2_max]
perceptron = nl.net.newp([dim1, dim2], num_output)

# Train the perceptron using the data
error_progress = perceptron.train(data, labels, epochs=100, show=20, lr=0.03)

# Plot the training progress
plt.figure()
plt.plot(error_progress)
plt.xlabel('Number of epochs')
plt.ylabel('Training error')
plt.title('Training error progress')
plt.grid()

plt.show()
```

Рис. 14.П1. Пример кода классификатора на основе перцептрона на Python

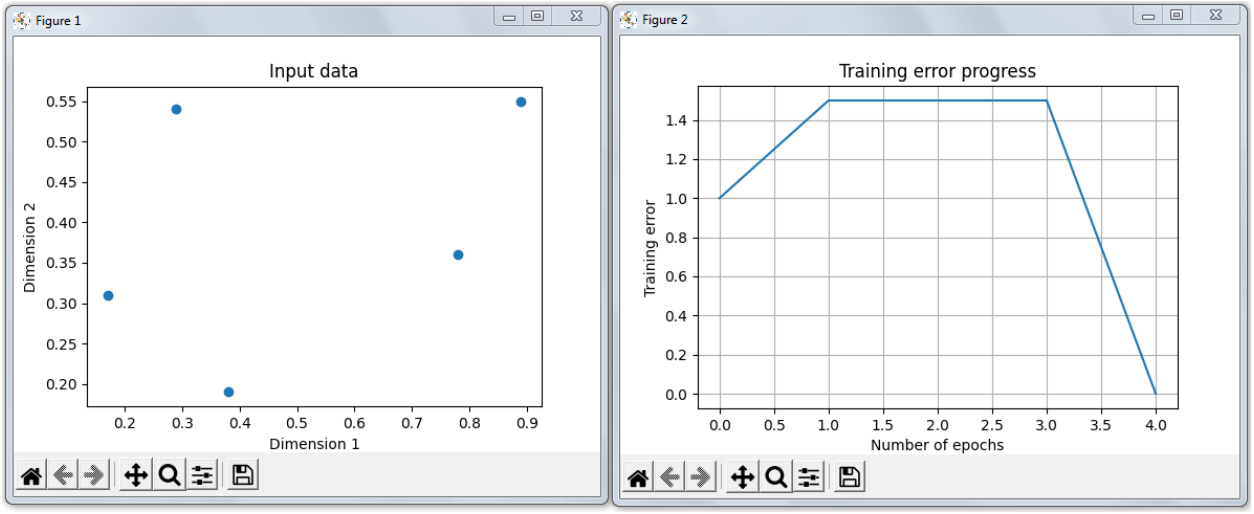

Рис. 14.П2. Входные точки данных и изменение ошибки обучения в зависимости от числа наблюдаемых эпох